

Web Engineering 1 | WE1

Summary

CONTENTS

1. HyperText Markup Language (HTML)	2
1.1. Content categories	2
1.2. Semantic elements	3
1.2.1. Webpage structure	3
1.2.2. h*	3
1.2.3. article	4
1.2.4. section	4
1.2.5. img	4
1.3. Void elements	4
2. Cascading Style Sheets (CSS)	4
2.1. Selectors	4
2.1.1. Specificity	5
2.2. Box-Model	6
2.3. Display	6
2.3.1. Flexbox	6
3. JavaScript (JS)	6
3.1. Atomic language constructs	7
3.1.1. Values and types	7
3.1.2. Functions	7
3.1.3. Objects	7
4. Document Object Model (DOM)	7

1. HYPERTEXT MARKUP LANGUAGE (HTML)

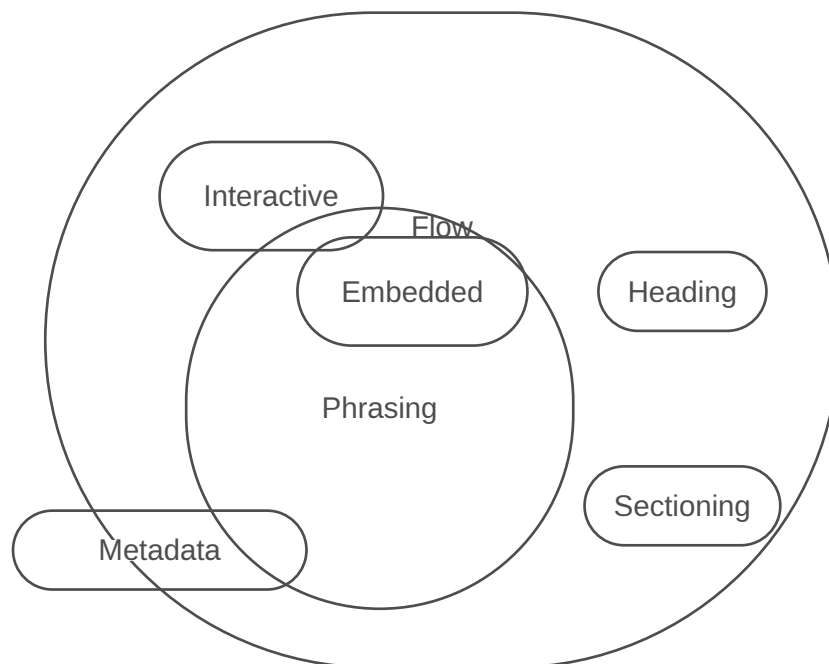
Structures the contents of a document, defines **what** is content and not **how** it looks.

<head> contains metadata, <body> contains the content.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <link rel="icon" href="ost-favicon-32x32.png">
  <title>Hello World</title>
</head>
<body>
  <h1>Hello World</h1>
  <p>Ich bin eine Testseite</p>
</body>
</html>
```

1.1. CONTENT CATEGORIES

Most HTML elements are a member of one or more content categories. The content categories are used to define the content model of elements, in other words, what each element can take as descendants.



Metadata: Elements belonging to the metadata content category modify the presentation or the behavior of the rest of the document, set up links to other documents, or convey other out-of-band information.

Flow: Flow content is a broad category that encompasses most elements that can go inside the <body> element, including heading elements, sectioning elements, phrasing elements, embedding elements, interactive elements, and form-related elements.

Sectioning: Creates a section in the current outline defining the scope of <header> and <footer> elements.

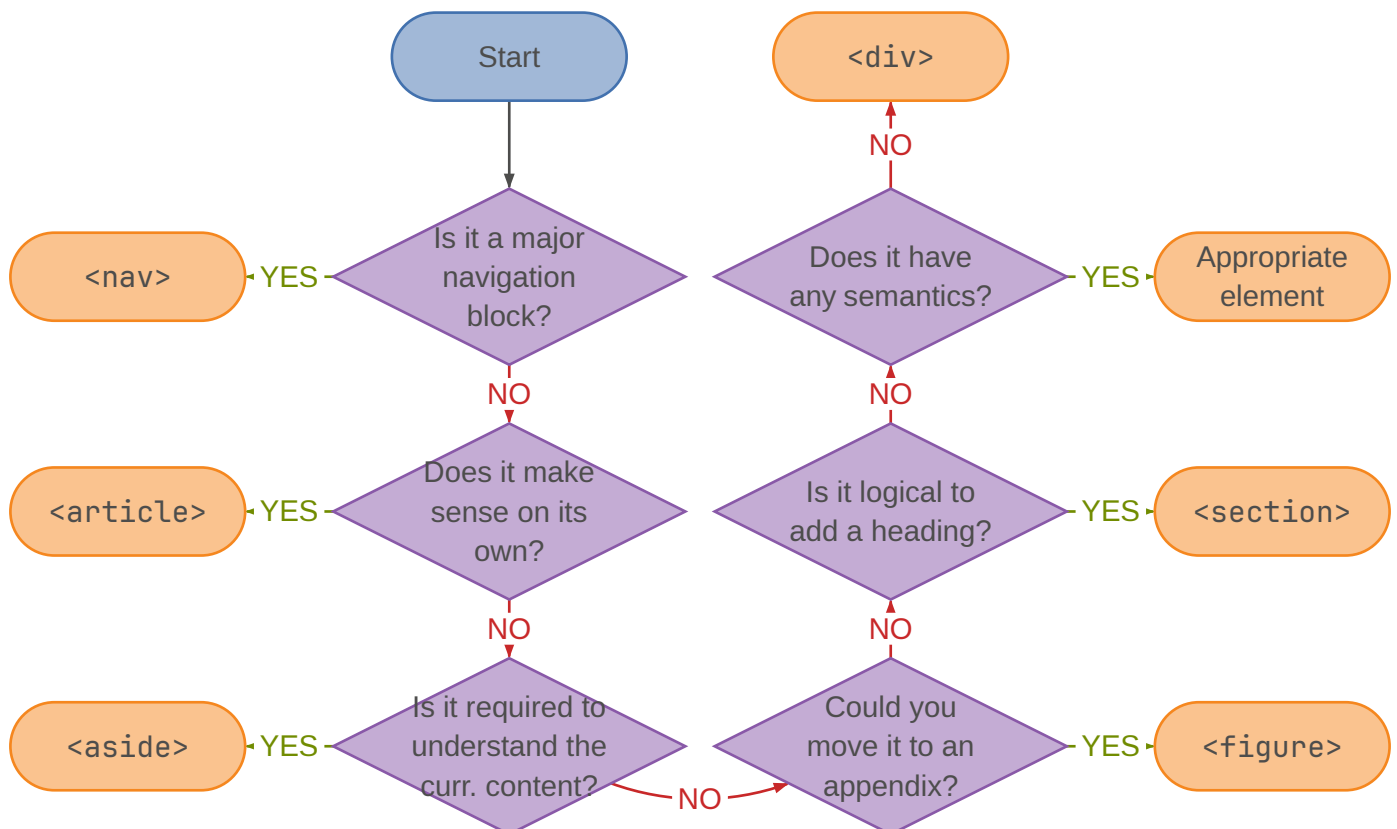
Heading: Defines the title of a section.

Phrasing: Refers to the text and the markup within a document. Sequences of phrasing content make up paragraphs.

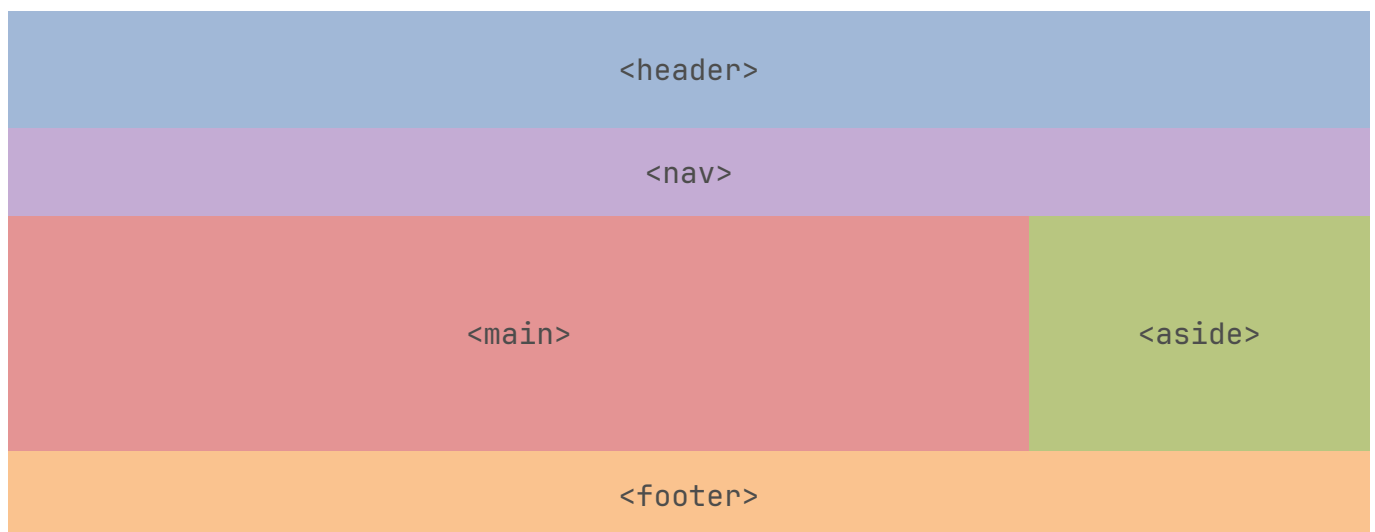
Embedded: Imports another resource or inserts content from another markup language or namespace into the document.

Interactive: Includes elements that are specifically designed for user interaction.

1.2. SEMANTIC ELEMENTS



1.2.1. Webpage structure



1.2.2. h*

`<h1>` should only appear once on the website.

1.2.3. article

A self-contained part of the document which is independently placed or should be reused.

```
<article>
  <section id="introduction">
    <p>JS was a mistake</p>
  </section>
  <section id="content">
    <!-- very long rant -->
  </section>
  <section id="summary">
    <p>JS was a mistake</p>
  </section>
</article>
```

1.2.4. section

General separation and grouping of content.

1.2.5. img

alt attribute's value should provide a clear and concise text replacement for the image's content. It should not describe the presence of the image itself or the file name of the image. The alt attribute **must be specified**.

If the image is of decorative nature only, one must still set alt, although empty: alt=""

1.3. VOID ELEMENTS

A void element is an element in HTML that cannot have any child nodes (i.e., nested elements or text nodes). Void elements only have a start tag; end tags must not be specified for void elements.

```
<area> <base> <br> <col> <embed> <hr> <img> <input> <link> <meta> <param>
<source> <track> <wbr>
```

TODO:

<https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements>

2. CASCADING STYLE SHEETS (CSS)

A simple mechanism for adding style (e.g., fonts, colors, spacing) to Web documents.

Inline: `<p style="color: red;">Text</p>` (Discouraged)

Inside of HTML: `<head><style> p { color: red; } </style></head>` (For small documents)

External file: `<link rel="stylesheet" href="styles.css">` (Standard)

2.1. SELECTORS

Class: `.classname` reusable for multiple elements

ID: `#idname` for unique, single element

TODO:

>
*
“ “
[]
&
.
#

```
<div id="div-1" class="recent significant good-to-go">1</div>
```

```
.recent // overrides others
```

2.1.1. Specificity

- 1) `!important`
- 2) `style=""` Attribute
- 3) (1) ID-Selectors
- 4) (2) Class-Selectors, Pseudoclasses (eg. `:hover`) and Attributes (`[ ... = ... ]`)
- 5) (3) Type-Selectors and Pseudoelements (eg. `::first-line`)

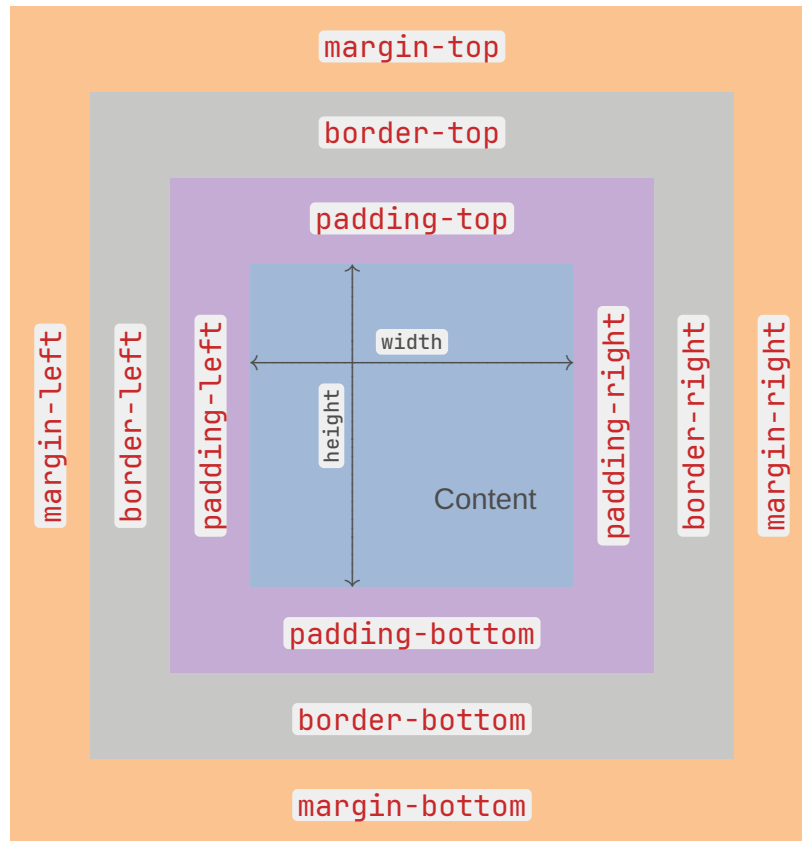
Example 1:

```
div.help > div#nested a:hover::before { }
```

Specificity: 124

1 id, 2 (pseudo-)classes, 4 (pseudo-)elements

2.2. BOX-MODEL



2.3. DISPLAY

Term	Definition
Block (p , h1 , u1)	Vertical alignment, width fills parent, height from content
Inline (a , strong)	Horizontal alignment, width and height from content, left/right margin/padding allowed, not top/bottom
Inline-Block	Flows like inline, but width/height margin/padding all allowed

`display: none;` Element isn't visible and doesn't take any space `visibility: hidden;` Element isn't visible but does use space

2.3.1. Flexbox

[flexbox.help](#)

3. JAVASCRIPT (JS)

[why did you do this to us, brendan eich](#)

3.1. ATOMIC LANGUAGE CONSTRUCTS

3.1.1. Values and types

JS is dynamically typed (help) and doesn't have constant values. It only has non-reassignable namespaces when using `const`. Using `let` you declare a reassignable variable and using `var` is a crime against humanity.

String interpolation:

```
let answer = 42;
console.log(`The answer is ${answer}`);
```

3.1.2. Functions

Functions are hoisted to the top. A function without an explicit return returns `undefined`.

```
function printName(name) {
  console.log(name);
}
```

Lambda functions

```
((a, b) => console.log(a + b === 5))(2, 3)
```

3.1.3. Objects

```
let obj = {
  key1: 69,
  key2: "nice",
};

let arr = [ obj, 1, "cursed" ];
```

TODO:

= vs. ≡

4. DOCUMENT OBJECT MODEL (DOM)